Experiment - 01

Name: Smyle Goyal

UID: 24BAI70469

Branch: BE-CSE(AIML)

Section: 24AIT_KRG-1_A

Semester: 5th

Subject: Advanced Database Management System

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1."Low Salary": All monthly salaries strictly less than $20,000.
2."Average Salary": All monthly salaries in the inclusive range [$20,000, $50,000].
3."High Salary": All monthly salaries strictly greater than $50,000. The final result table must report all three categories, even if a category contains zero accounts.

Output Table

category	accounts_count
Low Salary	1
Average Salary	1
High Salary	3

```
SELECT 'Low Salary' AS
    category, COUNT(*) AS
    accounts_count
FROM Accounts WHERE
 income < 20000 P
UNION ALL

 SELECT 'Average Salary' AS category,
    COUNT(*) AS accounts_count
FROM Accounts
WHERE income BETWEEN 20000 AND 50000

 UNION ALL

 SELECT 'High Salary' AS
    category, COUNT(*) AS
    accounts_count
FROM Accounts
WHERE income > 50000;
```

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1.Create and populate structural tracking tables for Departments, Employees, and Salaries.
2.Query the system using multi-table Joins to pull clear department breakdowns, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below $30,000.
3.List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Part B: Apply 10% HR increase using IN subquery filtering

Output Table:

name	dept_name	salary
Alice	HR	50000
Bob	HR	25000
Charlie	IT	80000
David	IT	40000
Emma	Sales	45000

Updated `Salaries` State Output Table:

emp_id	salary
1	55000
2	27500
3	80000
4	40000
5	45000

```
SELECT
  e.name,
  d.dept_name,
  s.salary
FROM Employees e
JOIN Departments
d
ON e.dept_id = d.dept_id
JOIN Salaries s
ON e.emp_id = s.emp_id;
```

```
UPDATE Salaries
SET salary = salary * 1.10
WHERE emp_id IN (
  SELECT emp_id
  FROM Employees
  WHERE dept_id =
  (
    SELECT dept_id
    FROM
    Departments
    WHERE dept_name = 'HR'
  )
);
```

```
DELETE FROM Salaries
WHERE salary <
30000;
```

```
SELECT
  e.name
  ,
  s.salary
FROM Employees e
JOIN Salaries s
ON e.emp_id = s.emp_id
WHERE s.salary > (
  SELECT
  AVG(salary) FROM
  Salaries
);
```

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.
Every customer must choose **exactly 3 different toppings** to prepare a pizza.
Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.
Requirements
Every pizza must contain **exactly 3 different toppings**.
The same topping **cannot appear more than once** in a pizza.
Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage
Pepperoni, Chicken, Sausage
Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.
Display the total ingredient cost of each pizza.
Sort the output:

First by **highest total cost**
If two pizzas have the same cost, sort them alphabetically.

Output Table

pizza	total_cost
Chicken,Pepperoni,Sausage	1.75
Extra Cheese,Pepperoni,Sausage	1.60
Chicken,Extra Cheese,Sausage	1.65
Extra Cheese,Onions,Sausage	1.35
Chicken,Onions,Sausage	1.50
Chicken,Extra Cheese,Pepperoni	1.45
Extra Cheese,Onions,Pepperoni	1.15
Chicken,Onions,Pepperoni	1.30
Chicken,Extra Cheese,Onions	1.20
Extra Cheese,Onions,Sausage	1.35

```
SELECT
CONCAT(t1.topping_name, ',', t2.topping_name, ',', t3.topping_name) AS pizza,
    ROUND(t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost, 2) AS total_cost
FROM pizza_toppings t1
  JOIN pizza_toppings t2
    ON t1.topping_name < t2.topping_name
  JOIN pizza_toppings t3
    ON t2.topping_name < t3.topping_name
  ORDER BY
total_cost DESC, pizza
    ASC;
```

(D)

Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase. Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

Requirements

1. Compare purchases **made by the same user only**.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur **after** the first purchase.
4. Same-day purchases must **not** be considered.
5. The second purchase must occur within **7 days** of the first purchase.
6. Display only the **unique User IDs** that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

Output Table

user_id
101
103

```
SELECT DISTINCT t1.user_id
  FROM Transactions t1
JOIN Transactions t2
ON t1.user_id = t2.user_id
AND t2.transaction_date > t1.transaction_date
    AND t2.transaction_date <= t1.transaction_date + INTERVAL '7 days'
 ORDER BY t1.user_id;
```